

# LEARNING WHEN TO SHARE: ADAPTIVE CHANNEL-HORIZON SHARING FOR LONG-TERM TIME-SERIES FORECASTING

Shiwei Pu<sup>a</sup>, Yuanjian Zhang<sup>a</sup>, Jingwei Luo<sup>a</sup>, Xiang Xu<sup>a</sup>, Yuxun Wu<sup>a</sup>, Chuanchang Liu<sup>a,\*</sup>

<sup>a</sup> Beijing University of Posts and Telecommunications, Beijing, China

## ABSTRACT

Long-term time-series forecasting requires models to capture common temporal patterns while accommodating differences among variables. We propose GateLinear, a lightweight framework that combines a shared prediction head with a channel-specific low-rank head. A shared multilayer perceptron extracts temporal features, and the individual head adapts a common low-rank basis through variable-specific coefficients. An aware feature gate (AF-Gate) uses the input context to construct channel-horizon fusion weights from multi-head outer products, allowing the balance between the two predictions to vary across samples, variables, and forecast steps. Experiments on 13 benchmarks show competitive accuracy with a compact model. Component ablations indicate that both prediction branches contribute to performance and that learned fusion improves on direct addition. These results support adaptive parameter sharing as a useful design for long-term forecasting.

**Index Terms**— Time-series forecasting, channel sharing, low-rank adaptation, adaptive gating

## 1. INTRODUCTION

Long-term time-series forecasting (LTSF) supports applications in energy, transportation, and weather. Forecasting architectures range from Autoformer [1] and FEDformer [2] to DLinear [3]. Two distinct design choices are parameter sharing and cross-variable interaction. PatchTST [4] processes variables independently with a shared encoder, whereas iTransformer [5] applies attention across variables. Independent prediction heads provide individualized mappings, with a parameter count proportional to the number of variables. Differences in dynamics, scale, and periodicity motivate a predictor that can adjust the balance between shared structure and variable-specific behavior.

A common predictor reuses temporal structure, while independent heads allow greater specialization at a higher parameter cost. Mixture-of-experts models provide a related approach by learning input-dependent combinations of expert predictions [6]. GateLinear applies this idea to shared and individualized forecasting: a shared dense head and a channel-modulated low-rank head produce two forecasts, and an input-conditioned gate assigns their weights at each channel-horizon pair. The framework makes three contributions. **First**, the dual-branch predictor combines a shared projection with variable-specific adaptation through a common low-rank basis, channel coefficients, and biases. **Second**, AF-Gate forms a channel-horizon mask from multiple outer products, providing sample-conditioned convex fusion with factorized gate outputs. **Third**, experiments on 13 benchmarks evaluate prediction accuracy and parameter count, while component ablations and visualizations examine the roles of the branches and the learned fusion weights.

## 2. RELATED WORK

Crossformer [7] models temporal and cross-variable dependencies, while iTransformer and FEDformer use variable attention and frequency decomposition, respectively. Channel-independent alternatives include PatchTST, DLinear, and FITS [8]; RevIN [9] addresses distribution shift through reversible normalization. CycleNet [10] learns periodic components, SparseTSF [11] models sparse cross-period dependencies, and XLinear [12] incorporates exogenous inputs into an MLP. TimeXer [13] and CARD [14] use attention to combine global and variable-specific information. Low-rank adaptation [15, 16, 17] specializes shared representations with few additional parameters. GateLinear follows this principle in its individual prediction head and learns how to combine that head with a shared dense predictor.

## 3. METHODOLOGY

### 3.1. Input representation and feature extraction

As shown in Fig. 1, a shared MLP feeds two prediction branches, which AF-Gate adaptively fuses. For one sample,  $\mathbf{X} \in \mathbb{R}^{L \times N}$  contains  $L$  past values of  $N$  variables;  $\mathbf{Y} \in \mathbb{R}^{P \times N}$  is the horizon- $P$  target. Widths  $D, E, K, R$  denote backbone hidden, feature, rank, and gate dimensions;  $M$  is the head count. Affine maps act on the last dimension;  $\odot$  denotes broadcast element-wise multiplication.

RevIN stores the per-variable input mean  $\mu$  and variance  $\sigma^2$ , then computes

$$\tilde{\mathbf{X}} = \gamma \odot \frac{\mathbf{X} - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad (1)$$

where  $\tilde{\mathbf{X}}$  is the normalized input,  $\gamma, \beta \in \mathbb{R}^N$  are learned scale and shift, and  $\epsilon = 10^{-5}$  ensures numerical stability. The timestamp representation  $\mathbf{T} \in \mathbb{R}^{L \times N}$  combines learned  $N$ -dimensional hour and quarter-hour embeddings:

$$\mathbf{T}_{t,:} = \text{Emb}_h(h_t) + \mathbb{I}_{\text{minute}} \text{Emb}_m(m_t). \quad (2)$$

At time step  $t$ ,  $h_t \in \{0, \dots, 23\}$  and  $m_t \in \{0, \dots, 3\}$  index the hour and quarter-hour;  $\mathbb{I}_{\text{minute}}$  is 1 for minute-level data and 0 otherwise. A shared two-layer MLP maps each variable's history through widths  $D$  and  $E$ :

$$\mathbf{Z} = \text{Linear}(\text{GELU}(\text{Linear}((\tilde{\mathbf{X}} + \mathbf{T})^\top))). \quad (3)$$

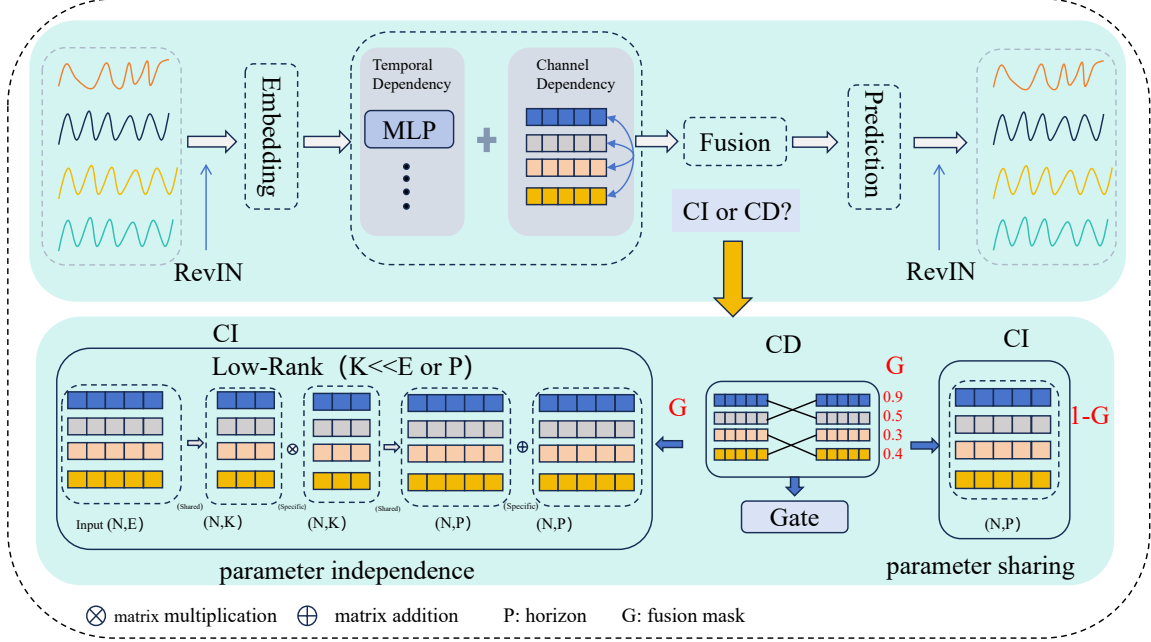
The resulting  $\mathbf{Z} \in \mathbb{R}^{N \times E}$  is refined by feature scores

$$\mathbf{a} = \text{Softmax}_E(\text{Linear}_{N \rightarrow 1}(\tanh(\mathbf{Z}^\top))), \quad (4)$$

$$\bar{\mathbf{Z}} = \mathbf{Z} + \mathbf{Z} \odot \mathbf{a}^\top. \quad (5)$$

Here  $\mathbf{a} \in \mathbb{R}^E$  contains feature weights,  $\text{Softmax}_E$  normalizes over features, and  $\bar{\mathbf{Z}} \in \mathbb{R}^{N \times E}$  is the refined representation. Superscript  $\top$  denotes transpose and  $:$  selects all entries along an axis. The bias-free scoring map aggregates variables using  $N$  weights and  $O(NE)$

\*Corresponding author: lcc3265@bupt.edu.cn.



**Fig. 1.** GateLinear overview. A variable-wise MLP feeds shared and channel-modulated heads; AF-Gate fuses their forecasts before inverse RevIN. CI/CD denote channel-independent/channel-dependent processing; the shared head shares parameters.

**Table 1.** Average results over horizons  $P \in \{96, 192, 336, 720\}$ . Bold and underline denote the lowest and second-lowest point estimates, respectively.

| Dataset     | GateLinear   |              | XLinear      |              | iTransformer |       | DLinear      |              | PatchTST     |       | CrossGNN     |              | MSGNet       |              | Leddam |       |
|-------------|--------------|--------------|--------------|--------------|--------------|-------|--------------|--------------|--------------|-------|--------------|--------------|--------------|--------------|--------|-------|
|             | MSE          | MAE          | MSE          | MAE          | MSE          | MAE   | MSE          | MAE          | MSE          | MAE   | MSE          | MAE          | MSE          | MAE          | MSE    | MAE   |
| Weather     | <b>0.243</b> | <b>0.271</b> | <u>0.251</u> | <u>0.275</u> | 0.286        | 0.301 | 0.266        | 0.317        | 0.268        | 0.288 | 0.266        | 0.300        | 0.255        | 0.286        | 0.264  | 0.303 |
| Electricity | <u>0.177</u> | <u>0.271</u> | <b>0.174</b> | <b>0.268</b> | 0.209        | 0.293 | 0.209        | 0.295        | 0.188        | 0.275 | 0.220        | 0.306        | 0.202        | 0.307        | 0.217  | 0.297 |
| Traffic     | <b>0.482</b> | <b>0.311</b> | 0.504        | <u>0.330</u> | 0.503        | 0.342 | 0.625        | 0.384        | 0.538        | 0.358 | 0.645        | 0.385        | 0.683        | 0.406        | 0.641  | 0.396 |
| Solar       | <b>0.236</b> | <b>0.268</b> | 0.256        | <u>0.276</u> | 0.308        | 0.316 | 0.327        | 0.399        | 0.270        | 0.309 | <u>0.247</u> | 0.317        | 0.275        | 0.306        | 0.366  | 0.364 |
| ETTh1       | <u>0.453</u> | <u>0.439</u> | <b>0.448</b> | <b>0.435</b> | 0.471        | 0.459 | <u>0.453</u> | 0.447        | 0.506        | 0.477 | 0.502        | 0.478        | 0.515        | 0.502        | 0.630  | 0.540 |
| ETTh2       | <u>0.385</u> | <u>0.407</u> | <b>0.378</b> | <b>0.401</b> | <u>0.385</u> | 0.409 | 0.532        | 0.501        | 0.398        | 0.417 | 0.399        | 0.420        | 0.415        | 0.423        | 0.418  | 0.433 |
| ETTm1       | <u>0.397</u> | <u>0.399</u> | <b>0.395</b> | <b>0.397</b> | 0.434        | 0.428 | 0.403        | 0.406        | <b>0.395</b> | 0.402 | 0.443        | 0.432        | 0.475        | 0.460        | 0.548  | 0.484 |
| ETTm2       | <u>0.284</u> | <b>0.326</b> | <b>0.282</b> | <b>0.326</b> | 0.295        | 0.338 | 0.339        | 0.391        | 0.286        | 0.331 | 0.285        | <u>0.327</u> | 0.297        | 0.334        | 0.301  | 0.344 |
| PEMS03      | <b>0.281</b> | <b>0.361</b> | <u>0.334</u> | <u>0.394</u> | 0.421        | 0.463 | 0.438        | 0.491        | 0.353        | 0.402 | 0.368        | 0.443        | 0.357        | 0.415        | 0.891  | 0.695 |
| PEMS04      | <b>0.240</b> | <b>0.343</b> | <u>0.265</u> | <u>0.362</u> | 0.576        | 0.561 | 0.428        | 0.480        | 0.461        | 0.467 | 0.424        | 0.480        | 0.369        | 0.448        | 0.953  | 0.731 |
| PEMS07      | <b>0.223</b> | <b>0.318</b> | 0.246        | 0.336        | 0.405        | 0.459 | 0.559        | 0.527        | 0.570        | 0.544 | 0.328        | 0.415        | <u>0.233</u> | <u>0.331</u> | 0.982  | 0.739 |
| PEMS08      | <b>0.426</b> | <b>0.393</b> | <u>0.482</u> | <u>0.431</u> | 0.713        | 0.566 | 0.666        | 0.539        | 0.775        | 0.584 | 0.533        | 0.482        | <u>0.505</u> | <u>0.437</u> | 1.145  | 0.766 |
| Exchange    | 0.389        | 0.418        | <u>0.382</u> | <u>0.415</u> | 0.394        | 0.428 | <b>0.377</b> | <b>0.405</b> | 0.392        | 0.420 | 0.413        | 0.423        | 0.471        | 0.468        | 0.437  | 0.438 |

operations. The residual scales feature  $e$  by  $1 + a_e \in (1, 2)$ , preserving the representation while emphasizing selected features. We assess refinement in the ablation study.

### 3.2. Dual-branch prediction and low-rank analysis

The shared branch applies the same projection to every variable:

$$\mathbf{Y}^{\text{sh}} = \bar{\mathbf{Z}}\mathbf{W}^{\text{sh}} + \mathbf{b}^{\text{sh}}, \quad (6)$$

where  $\mathbf{W}^{\text{sh}} \in \mathbb{R}^{E \times P}$  and  $\mathbf{b}^{\text{sh}} \in \mathbb{R}^P$  are projection weights and bias. Both branch outputs  $\mathbf{Y}^{\text{sh}}, \mathbf{Y}^{\text{ind}} \in \mathbb{R}^{N \times P}$  are forecasts on the normalized scale. The shared head uses  $EP + P$  parameters; cross-variable context comes from feature refinement and the pooled gate.

The individual branch instead shares factors  $\mathbf{W}_1 \in \mathbb{R}^{E \times K}$  and

$\mathbf{W}_2 \in \mathbb{R}^{K \times P}$ , with channel-specific coefficients  $\mathbf{S} \in \mathbb{R}^{N \times K}$  and biases  $\mathbf{B} \in \mathbb{R}^{N \times P}$ :

$$\mathbf{Y}^{\text{ind}} = ((\bar{\mathbf{Z}}\mathbf{W}_1) \odot \mathbf{S})\mathbf{W}_2 + \mathbf{B}. \quad (7)$$

For variable  $n$ , the effective projection is

$$\mathbf{W}_n^{\text{eff}} = \sum_{k=1}^K S_{nk} \mathbf{W}_{1,:k} \mathbf{W}_{2,k,:}, \quad \text{rank}(\mathbf{W}_n^{\text{eff}}) \leq K. \quad (8)$$

For fixed learned factors, the variable-specific projections lie in the span of at most  $K$  shared rank-one matrices. Each variable learns the amplitude and sign of these components, together with its own bias. The common basis constrains the available projection directions, while the coefficients provide an economical form of specialization. Compared with independent rank- $K$  factors, this design ex-

**Table 2.** Forecasting results for different datasets and horizons  $P$ . Bold and underline denote the lowest and second-lowest point estimates, respectively.

| Dataset     | $P$ | GateLinear   |              | XLinear      |              | iTransformer |       | DLinear      |       | PatchTST     |              | CrossGNN     |       | MSGNet |       | Leddham |       |
|-------------|-----|--------------|--------------|--------------|--------------|--------------|-------|--------------|-------|--------------|--------------|--------------|-------|--------|-------|---------|-------|
|             |     | MSE          | MAE          | MSE          | MAE          | MSE          | MAE   | MSE          | MAE   | MSE          | MAE          | MSE          | MAE   | MSE    | MAE   | MSE     | MAE   |
| Weather     | 96  | <b>0.159</b> | <b>0.204</b> | <u>0.167</u> | <u>0.210</u> | 0.212        | 0.244 | 0.196        | 0.256 | 0.190        | 0.231        | 0.184        | 0.236 | 0.169  | 0.222 | 0.181   | 0.239 |
|             | 192 | <b>0.207</b> | <b>0.248</b> | <u>0.216</u> | <u>0.253</u> | 0.252        | 0.284 | 0.236        | 0.292 | 0.231        | 0.264        | 0.232        | 0.280 | 0.218  | 0.264 | 0.225   | 0.272 |
|             | 336 | <b>0.263</b> | <b>0.290</b> | <u>0.271</u> | <u>0.293</u> | 0.304        | 0.314 | 0.282        | 0.333 | 0.288        | 0.304        | 0.285        | 0.319 | 0.281  | 0.306 | 0.281   | 0.314 |
|             | 720 | <b>0.342</b> | <b>0.343</b> | 0.351        | <u>0.344</u> | 0.375        | 0.361 | <u>0.349</u> | 0.386 | 0.363        | 0.352        | 0.363        | 0.366 | 0.354  | 0.353 | 0.370   | 0.388 |
| Electricity | 96  | <b>0.145</b> | <b>0.242</b> | 0.145        | <u>0.243</u> | 0.181        | 0.268 | 0.195        | 0.278 | <u>0.161</u> | 0.248        | 0.201        | 0.285 | 0.170  | 0.280 | 0.197   | 0.276 |
|             | 192 | <b>0.161</b> | <b>0.254</b> | <u>0.162</u> | <u>0.257</u> | 0.190        | 0.277 | 0.194        | 0.280 | 0.173        | 0.261        | 0.205        | 0.293 | 0.192  | 0.300 | 0.199   | 0.282 |
|             | 336 | <u>0.179</u> | 0.274        | <b>0.178</b> | <b>0.274</b> | 0.210        | 0.295 | 0.207        | 0.295 | 0.190        | <u>0.278</u> | 0.217        | 0.307 | 0.208  | 0.315 | 0.214   | 0.297 |
|             | 720 | <u>0.222</u> | 0.313        | <b>0.210</b> | <b>0.301</b> | 0.256        | 0.333 | 0.242        | 0.328 | 0.229        | <u>0.312</u> | 0.258        | 0.341 | 0.237  | 0.334 | 0.258   | 0.334 |
| Traffic     | 96  | <b>0.456</b> | <b>0.297</b> | 0.475        | <u>0.315</u> | <u>0.473</u> | 0.327 | 0.650        | 0.398 | 0.528        | 0.357        | 0.659        | 0.398 | 0.617  | 0.363 | 0.653   | 0.398 |
|             | 192 | <b>0.465</b> | <b>0.303</b> | <u>0.481</u> | <u>0.320</u> | 0.484        | 0.331 | 0.598        | 0.371 | 0.525        | 0.353        | 0.610        | 0.377 | 0.653  | 0.392 | 0.612   | 0.381 |
|             | 336 | <b>0.478</b> | <b>0.312</b> | <u>0.499</u> | <u>0.322</u> | 0.506        | 0.342 | 0.605        | 0.373 | 0.534        | 0.354        | 0.618        | 0.382 | 0.689  | 0.414 | 0.622   | 0.386 |
|             | 720 | <b>0.527</b> | <b>0.333</b> | 0.559        | <u>0.363</u> | <u>0.549</u> | 0.368 | 0.645        | 0.395 | 0.565        | 0.366        | 0.694        | 0.383 | 0.771  | 0.457 | 0.679   | 0.418 |
| Solar       | 96  | <b>0.206</b> | <b>0.246</b> | <u>0.212</u> | <u>0.248</u> | 0.254        | 0.284 | 0.288        | 0.377 | 0.239        | 0.291        | 0.220        | 0.305 | 0.228  | 0.269 | 0.313   | 0.337 |
|             | 192 | <b>0.234</b> | <b>0.269</b> | 0.249        | <u>0.270</u> | 0.301        | 0.312 | 0.316        | 0.394 | 0.266        | 0.305        | <u>0.244</u> | 0.320 | 0.269  | 0.303 | 0.347   | 0.368 |
|             | 336 | <b>0.255</b> | <b>0.283</b> | 0.277        | <u>0.289</u> | 0.339        | 0.332 | 0.349        | 0.412 | 0.290        | 0.325        | <u>0.265</u> | 0.327 | 0.302  | 0.323 | 0.399   | 0.378 |
|             | 720 | <b>0.248</b> | <b>0.276</b> | 0.287        | <u>0.296</u> | 0.337        | 0.337 | 0.354        | 0.412 | 0.286        | 0.314        | <u>0.258</u> | 0.316 | 0.303  | 0.329 | 0.403   | 0.373 |

changes freedom to choose a separate basis for lower parameter cost. We set

$$K = \max \left( 1, \left\lfloor \alpha \frac{EP}{E+P} \right\rfloor \right), \quad \alpha = 0.3. \quad (9)$$

Here  $\alpha$  controls the rank relative to  $E$  and  $P$ . The head uses  $C_{\text{LR}} = K(E+P) + N(K+P)$  parameters, compared with  $C_{\text{full}} = N(EP+P)$  for independent dense heads. The removed fraction is  $1 - C_{\text{LR}}/C_{\text{full}}$ ; using  $K(E+P) \approx \alpha EP$  gives

$$\frac{C_{\text{LR}}}{C_{\text{full}}} \approx \frac{\alpha E}{N(E+1)} + \frac{K+P}{P(E+1)}. \quad (10)$$

On Electricity ( $E = 512, N = 321$ ), horizons 96, 192, 336, 720 yield ranks 24, 41, 60, 89 and head sizes (millions) 0.053, 0.104, 0.178, 0.369 versus 15.81, 31.62, 55.33, 118.56 for dense individual heads. These counts correspond to reductions of 99.66–99.69% for the individual head. Independent low-rank factors would require  $N(EK + KP + P)$  parameters. Sharing the factors concentrates the variable-specific parameter cost in the  $K$  coefficients and  $P$  biases.

### 3.3. Aware feature gate and training objective

AF-Gate pools the refined features to form a sample context and a compact representation:

$$\mathbf{g} = N^{-1} \sum_{n=1}^N \bar{\mathbf{z}}_{n,:} \in \mathbb{R}^E, \quad (11)$$

$$\mathbf{h} = \text{Dropout}(\text{GELU}(\text{Linear}(\mathbf{g}))) \in \mathbb{R}^R. \quad (12)$$

Separate affine projections followed by sigmoid and reshaping produce channel factors  $\mathbf{G}^c \in [0, 1]^{M \times N}$  and horizon factors  $\mathbf{G}^p \in [0, 1]^{M \times P}$ . Each head forms an outer product; learned logits  $\boldsymbol{\rho} \in \mathbb{R}^M$  yield normalized head weights  $\boldsymbol{\pi}$ :

$$\mathbf{G} = \sum_{m=1}^M \pi_m (\mathbf{G}_{m,:}^c)^\top \mathbf{G}_{m,:}^p, \quad \boldsymbol{\pi} = \text{Softmax}(\boldsymbol{\rho}). \quad (13)$$

The mask  $\mathbf{G} \in [0, 1]^{N \times P}$  has rank at most  $M$ . Each head represents a separable channel–horizon preference, and their weighted combination captures multiple such patterns. The projections produce

$M(N+P)$  factor values, with  $O(MNP)$  operations to assemble the mask. The input context  $\mathbf{g}$  makes the factors sample-dependent, while  $\boldsymbol{\pi}$  provides globally learned head weights. At each channel–horizon pair, the two forecasts are combined using complementary weights:

$$\hat{\mathbf{Y}}^{\text{norm}} = \mathbf{G} \odot \mathbf{Y}^{\text{sh}} + (\mathbf{1} - \mathbf{G}) \odot \mathbf{Y}^{\text{ind}}. \quad (14)$$

Here  $\hat{\mathbf{Y}}^{\text{norm}}$  is the fused normalized forecast. RevIN restores  $\tilde{\mathbf{Y}} = (\hat{\mathbf{Y}}^{\text{norm}})^\top$  to the original scale:

$$\hat{\mathbf{Y}} = \frac{\tilde{\mathbf{Y}} - \boldsymbol{\beta}}{\boldsymbol{\gamma} + \epsilon^2} \odot \sqrt{\boldsymbol{\sigma}^2 + \epsilon} + \boldsymbol{\mu}, \quad (15)$$

$$\mathcal{L}_{\text{MSE}} = (PN)^{-1} \|\hat{\mathbf{Y}} - \mathbf{Y}\|_F^2. \quad (16)$$

Here  $\hat{\mathbf{Y}} \in \mathbb{R}^{P \times N}$  is the final forecast and  $\|\cdot\|_F$  is the Frobenius norm. Mean squared error  $\mathcal{L}_{\text{MSE}}$  is the sole training objective.

## 4. EXPERIMENTS

### 4.1. Setup and reproducibility

We evaluate on Weather, Electricity, Traffic, Solar, Exchange Rate, ETTh1/2, ETTm1/2 [18], and PEMS03/04/07/08 [19]. The look-back is  $L = 96$  and horizons are  $P \in \{96, 192, 336, 720\}$ . Chronological splits and normalization follow prior work. All reported runs share one repository, loader, splits, input lengths, horizons, and metrics. Baselines are XLinear [12], DLinear, PatchTST, iTransformer, CrossGNN [20], MSGNet [21], and Leddam [22]. We report MSE and mean absolute error (MAE); lower is better.

GateLinear uses PyTorch Adam ( $\beta_1 = 0.9, \beta_2 = 0.999, \epsilon_{\text{Adam}} = 10^{-8}$ ) without weight decay. Training runs for at most 30 epochs with early-stopping patience 5 and zero minimum-improvement threshold. Learning-rate/batch-size pairs are Weather: 0.001/64; Electricity: 0.005/16; ETT: 0.001/256; PEMS: 0.003/32; Solar: 0.003/64; Traffic: 0.003/16; Exchange: 0.005/8. Training shuffles and drops incomplete batches; validation/test retain order and all samples.

We set  $D = 128$ ,  $E = 512$ ,  $R = 16$ ,  $M = 8$ , gate dropout 0.1, and  $\alpha = 0.3$ . RevIN uses affine parameters and  $\epsilon = 10^{-5}$ . Experiments use seed 2027 for Python, NumPy, and PyTorch; the reported metrics are single-seed point estimates.

#### 4.2. Forecasting accuracy and parameter efficiency

Table 1 compares horizon-averaged results across 13 datasets. GateLinear achieves the lowest reported MSE and MAE on Weather, Traffic, Solar, and all four PEMS datasets. XLinear performs competitively on Electricity and ETT, while DLinear leads on Exchange. Table 2 presents per-horizon results for four datasets, where GateLinear’s advantage on Weather, Traffic, and Solar extends across the evaluated horizons. Table 3 compares prediction accuracy and

**Table 3.** Accuracy and parameters (M: millions) at  $L = P = 96$ . Bold and underline denote the lowest and second-lowest point estimates, respectively.

| Model        | Weather       |               |             | Electricity   |               |             |
|--------------|---------------|---------------|-------------|---------------|---------------|-------------|
|              | MSE           | MAE           | M           | MSE           | MAE           | M           |
| GateLinear   | <b>0.1595</b> | <b>0.2039</b> | 0.17        | <b>0.1454</b> | <u>0.2420</u> | 0.25        |
| CrossGNN     | 0.1848        | 0.2387        | <b>0.01</b> | 0.1911        | 0.2781        | <b>0.02</b> |
| DLinear      | 0.1956        | 0.2561        | 0.02        | 0.1947        | 0.2778        | <b>0.02</b> |
| iTransformer | 0.2118        | 0.2437        | 6.40        | 0.1474        | <b>0.2393</b> | 4.83        |
| Leddiam      | 0.1715        | 0.2347        | 16.96       | 0.1971        | 0.2756        | 16.97       |
| MSGNet       | 0.1682        | 0.2207        | 9.68        | 0.1704        | 0.2803        | 17.33       |
| PatchTST     | 0.1896        | 0.2305        | 6.91        | 0.1614        | 0.2491        | 5.34        |
| XLinear      | <u>0.1677</u> | <u>0.2119</u> | 0.16        | <u>0.1455</u> | 0.2427        | 0.32        |

trainable parameter counts at  $L = P = 96$ . On Weather, GateLinear achieves MSE/MAE of 0.1595/0.2039 with 0.17M parameters, compared with 0.1677/0.2119 and 0.16M for XLinear, reducing the errors by 4.89% and 3.76%. On Electricity, GateLinear obtains 0.1454/0.2420 with 0.25M parameters, compared with XLinear’s 0.1455/0.2427 and 0.32M. In this comparison, iTransformer obtains 0.1474/0.2393 with 4.83M parameters. GateLinear therefore achieves comparable errors with approximately 5.2% of iTransformer’s parameter count.

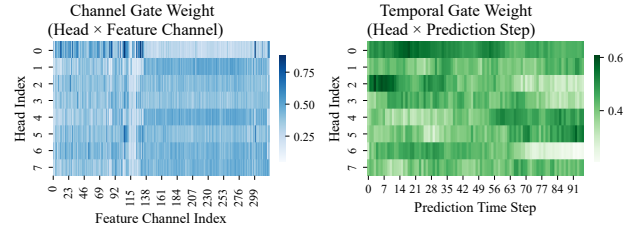
#### 4.3. Ablation and adaptive-sharing analysis

Table 4 evaluates the components at  $L = P = 96$ . Replacing AF-Gate with direct addition increases PEMS03 MSE from 0.2475 to 0.3418 (38.1%). Removing either prediction branch also increases the errors on all three datasets, supporting the contribution of both shared and individualized forecasts. The gate comparison evaluates the complete weighted-fusion mechanism: direct addition changes both the branch weights and the output scale. Separate controls would be needed to distinguish the contributions of sample, channel, and horizon adaptation. Feature refinement and temporal embeddings have dataset-dependent effects. Removing refinement reduces Electricity MSE/MAE from 0.1451/0.2416 to 0.1427/0.2372, while removing temporal embeddings reduces PEMS03 errors from 0.2475/0.3406 to 0.2277/0.3287. Their removal slightly increases errors on the other reported datasets. These results favor selecting the auxiliary modules using validation performance.

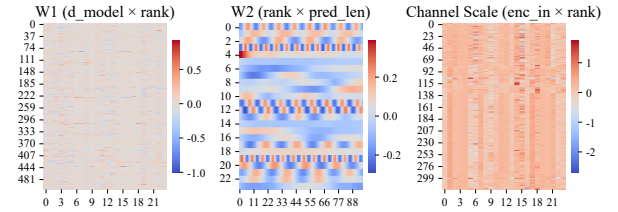
Fig. 2 shows distinct preferences across variables and forecast steps. Flat profiles favor similar fusion weights, while peaks emphasize particular channels or horizons. Learned head weights combine these profiles into the final fusion mask. Fig. 3 visualizes the shared factors and variable-specific coefficients. Similar coefficient patterns

**Table 4.** Ablation results ( $L = P = 96$ ). Temp. Emb., Shared Bra., and Indiv. Bra. denote temporal embedding, shared branch, and individual branch, respectively.

| Model           | Electricity   |               | PEMS03        |               | Weather       |               |
|-----------------|---------------|---------------|---------------|---------------|---------------|---------------|
|                 | MSE           | MAE           | MSE           | MAE           | MSE           | MAE           |
| Full Model      | 0.1451        | 0.2416        | <u>0.2475</u> | <u>0.3406</u> | <b>0.1595</b> | <b>0.2039</b> |
| w/o Attention   | <b>0.1427</b> | <b>0.2372</b> | 0.2493        | 0.3493        | 0.1604        | 0.2042        |
| w/o Temp. Emb.  | <u>0.1436</u> | <u>0.2400</u> | <b>0.2277</b> | <b>0.3287</b> | <u>0.1602</u> | 0.2049        |
| w/o Gate        | 0.1585        | 0.2526        | 0.3418        | 0.4066        | 0.1674        | 0.2124        |
| w/o Shared Bra. | 0.1597        | 0.2571        | 0.2661        | 0.3617        | 0.1611        | 0.2062        |
| w/o Indiv. Bra. | 0.1595        | 0.2529        | 0.3267        | 0.4025        | 0.1676        | 0.2125        |



**Fig. 2.** AF-Gate factors on Electricity ( $P = 96$ ), before head weighting. Rows index heads; columns index variables (left) or horizons (right).



**Fig. 3.** Shared factors  $W_1$ ,  $W_2$  and channel-specific coefficients  $S$  on Electricity. Plot labels  $d\_model$ ,  $rank$ ,  $pred\_len$ , and  $enc\_in$  correspond to  $E$ ,  $K$ ,  $P$ ,  $N$ , respectively.

indicate similar weightings of the common basis. Each channel adjusts the amplitude and sign of these components using  $K$  coefficients and adds a  $P$ -dimensional bias. The coefficients are shared across samples, while AF-Gate adapts the branch weights to each input. This separates persistent channel specialization from input-dependent sharing.

## 5. CONCLUSION

GateLinear combines shared prediction with low-rank adaptation for individual channels. AF-Gate uses the input context to balance the two forecasts across channels and prediction steps. Experiments on 13 benchmarks show competitive accuracy, and ablations support both branches and learned fusion. Sharing the projection factors limits the additional parameters per variable to modulation coefficients and biases, offering a compact approach to individualized long-term forecasting. Future work will examine how learned sharing patterns evolve under changes in temporal dynamics.

**AI-assisted language editing.** ChatGPT (OpenAI) was used only for language editing to improve clarity and conciseness; all content was reviewed and verified by the authors.

## 6. REFERENCES

- [1] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Advances in Neural Information Processing Systems*, 2021, vol. 34.
- [2] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin, “FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting,” in *Proceedings of the 39th International Conference on Machine Learning*, 2022, vol. 162 of *Proceedings of Machine Learning Research*.
- [3] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu, “Are transformers effective for time series forecasting?,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023, vol. 37, pp. 11121–11128.
- [4] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations*, 2023.
- [5] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long, “iTransformer: Inverted transformers are effective for time series forecasting,” in *International Conference on Learning Representations*, 2024.
- [6] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton, “Adaptive mixtures of local experts,” *Neural Computation*, vol. 3, no. 1, pp. 79–87, 1991.
- [7] Yunhao Zhang and Junchi Yan, “Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting,” in *International Conference on Learning Representations*, 2023.
- [8] Zhijian Xu, Ailing Zeng, and Qiang Xu, “Fits: Modeling time series with 10k parameters,” in *International Conference on Learning Representations*, 2024, vol. 2024, pp. 26295–26318.
- [9] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo, “Reversible instance normalization for accurate time-series forecasting against distribution shift,” in *International Conference on Learning Representations*, 2022.
- [10] Shengsheng Lin, Weiwei Lin, Xinyi Hu, Wentai Wu, Ruichao Mo, and Haocheng Zhong, “CycleNet: Enhancing time series forecasting through modeling periodic patterns,” in *Advances in Neural Information Processing Systems*, 2024.
- [11] Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang, “SparseTSF: Modeling long-term time series forecasting with 1k parameters,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, vol. 235 of *Proceedings of Machine Learning Research*.
- [12] Xinyang Chen, Huidong Jin, Yu Huang, and Zaiwen Feng, “XLinear: A lightweight and accurate MLP-based model for long-term time series forecasting with exogenous inputs,” *arXiv preprint arXiv:2601.09237*, 2026.
- [13] Yuxuan Wang, Haixu Wu, Jiayang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long, “TimeXer: Empowering transformers for time series forecasting with exogenous variables,” in *Advances in Neural Information Processing Systems*, 2024.
- [14] Tian Zhou, Qingsong Wen, Jinyang Gao, Bolin Ding, Rong Jin, et al., “Card: Channel aligned robust blend transformer for time series forecasting,” in *International conference on learning representations*, 2024, vol. 2024, pp. 11245–11283.
- [15] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [16] Dawid J. Kopiczko, Tijmen Blankevoort, and Yuki M. Asano, “Vera: Vector-based random matrix adaptation,” in *International Conference on Learning Representations*, 2024.
- [17] Tong Nie, Yuewen Mei, Guoyang Qin, Jian Sun, and Wei Ma, “Channel-aware low-rank adaptation in time series forecasting,” in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 3959–3963.
- [18] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021, vol. 35, pp. 11106–11115.
- [19] Minhao Liu, Ailing Zeng, Mu-Hwa Chen, Zhijian Xu, Qiuxia Lai, Lingna Ma, and Qiang Xu, “Scinet: Time series modeling and forecasting with sample convolution and interaction,” *Advances in Neural Information Processing Systems* 35, 2021.
- [20] Qihe Huang, Lei Shen, Ruixin Zhang, Shouhong Ding, Binwu Wang, Zhengyang Zhou, and Yang Wang, “CrossGNN: Confronting noisy multivariate time series via cross interaction refinement,” in *Advances in Neural Information Processing Systems*, 2023, vol. 36, pp. 46885–46902.
- [21] Wanlin Cai, Yuxuan Liang, Xianggen Liu, Jianshuai Feng, and Yuankai Wu, “MSGNet: Learning multi-scale inter-series correlations for multivariate time series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024, vol. 38, pp. 11141–11149.
- [22] Guoqi Yu, Jing Zou, Xiaowei Hu, Angelica I. Aviles-Rivero, Jing Qin, and Shujun Wang, “Revitalizing multivariate time series forecasting: Learnable decomposition with inter-series dependencies and intra-series variations modeling,” in *Proceedings of the 41st International Conference on Machine Learning*, 2024, vol. 235 of *Proceedings of Machine Learning Research*.